

PACT: Contract-Verified Receding-Horizon Planning for Compute-Bounded Tool Agents

Pu Yang

Independent Researcher
npuyangpu@gmail.com

Abstract

Tool agents often let one language model both interpret intent and authorize external effects. We instead present PACT, a contract-guided Track 2 agent that treats its Cerebras-hosted model as an *untrusted semantic compiler*. The model emits a compact strict-schema wire plan; a deterministic kernel then performs strict decoding, typed-DAG validation, live Draft 2020–12 JSON Schema checking, and evidence-aware execution. PACT exposes at most one outstanding action, recompiles short horizons as evidence arrives, scopes confirmation to an exact operation–argument–schema triple, and carries successful action obligations until a grounded terminal response. One verification-guided repair is permitted. Provider retries additionally obey finite retry and cumulative-wait budgets, while final A2A text responses report prompt, completion, and thinking-token metrics. We state guarantees only for local structure, contracts, transitions, and evidence—not for the model’s interpretation of natural language. At report freeze, 330 tests plus three subtests pass, and one public smoke task completes end to end. We treat that single trajectory as engineering evidence, not a performance estimate.

1 Introduction

CAR-bench evaluates whether tool agents act consistently and respect limits under ambiguous or incomplete information [Kirmayr *et al.*, 2026]. Through the Agent2Agent (A2A) protocol [A2A Protocol Working Group, 2026], the evaluator supplies policy text, messages, live tool declarations, and tool results; it alone executes external operations. Track 2 additionally rewards useful behavior under a bounded Cerebras inference route. A ReAct-style loop [Yao and others, 2023] is flexible, but the same stochastic component commonly interprets the request, constructs arguments, decides that preconditions hold, and declares success.

PACT separates those responsibilities. The LLM translates language into a small obligation program, but has no execution authority. A local monitor admits the program only after checking its syntax, graph, current capability contracts, resource

bounds, confirmations, and evidence dependencies. This resembles proof-carrying code’s separation of an untrusted producer from a small checker [Necula, 1997] and runtime verification’s monitoring of concrete executions [Leucker and Schallhart, 2009]; the “proof” here is deliberately narrower, consisting of checkable structural and provenance conditions.

Our contributions are: (i) a provider-compatible, strict wire format that decouples structured generation from dynamic tool schemas; (ii) a typed PlanIR and receding-horizon obligation runtime with evidence-grounded outcomes, one-shot confirmation, and contract revalidation at emission; and (iii) a precise assurance boundary plus reproducible failure-oriented tests and compute telemetry. The trusted kernel is domain-agnostic: it contains no task IDs, benchmark utterances, or automotive workflow branches.

Positioning. PACT is neither a direct model-to-tool interface nor a catalog of benchmark-shaped workflows. Direct tool calling constrains the surface form of an action, but syntactically valid JSON does not establish that the named operation is still available, its arguments satisfy a changing recursive schema, a confirmation applies to this exact effect, or a result supports the claimed terminal outcome. Empirical work on structured generation similarly finds a gap between nominal schema conformance and reliable generation across real schemas [Geng *et al.*, 2025]. PACT therefore uses two languages: a small provider-facing wire language optimized for constrained decoding, and a richer local PlanIR checked against the concrete runtime state. Conversely, unlike a hand-authored finite-state policy, the PlanIR is synthesized anew from the current policy, request, evidence, and capabilities. Selectivity occurs at each verified plan horizon: the model proposes semantics, while the kernel owns only transitions that are structurally determined by the accepted graph.

2 PACT Architecture

2.1 State and Untrusted Compilation

For one A2A context q , PACT maintains

$$S_t^q = (P, g, H_t, T_t, E_t, G_t, p_t, O_t, M_t), \quad (1)$$

where P is the evaluator-supplied policy, g the current goal, H_t a bounded conversation, T_t the live tool contracts, E_t an append-only evidence ledger, G_t the accepted plan, p_t its single pending action, O_t carried completion obligations,

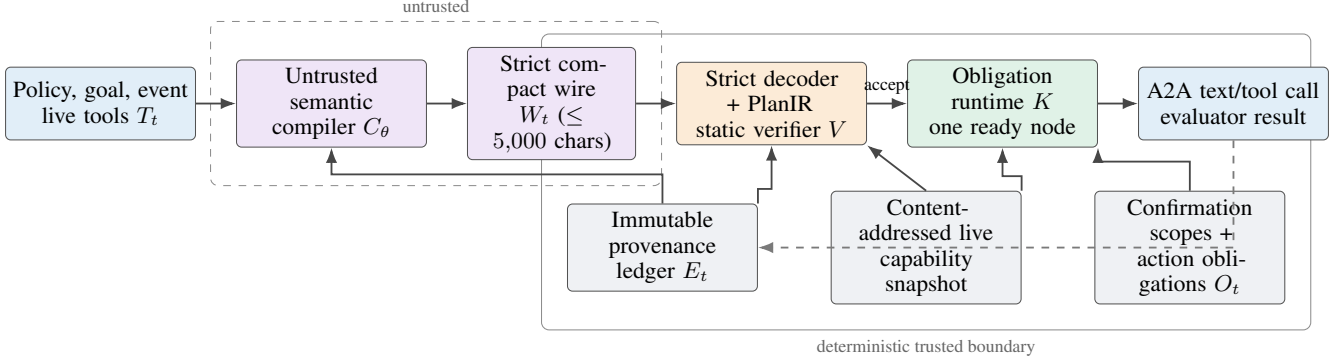


Figure 1: PACT separates semantic proposal from execution authority. The model can only produce a wire candidate. Local validation yields a typed plan, and the runtime emits one externally visible obligation at a time. Results enter future decisions only through the provenance ledger; the live capability snapshot is checked both before plan admission and immediately before call emission.

and M_t accumulated inference metrics. Context cancellation deletes this state.

The semantic compiler C_θ receives P in a separate trusted system partition and the goal, recent events, T_t , and a typed projection of E_t as task state. It proposes a wire plan

$$W_t = C_\theta(P, g, H_t, T_t, \text{view}(E_t), O_t). \quad (2)$$

The provider schema is strict: every object forbids extra properties, and its canonical serialization is capped at 5,000 characters to match the Cerebras structured-output contract [Cerebras Systems, 2026]. Dynamic arguments cannot be represented as an unrestricted object under that contract, so each operation node carries an `arguments-json` string. Locally, PACT rejects duplicate keys and non-finite numbers, decodes that string to one JSON object, and validates it against the actual live schema. Thus provider constrained decoding is an early filter, not the authority. If local checking rejects the first proposal, one repair prompt contains value-redacted typed issues; a second rejection terminates safely. The submitted default is `gpt-oss-120b` at low reasoning effort, leaving the 4,096-token output budget primarily for the structured plan. A provider `length` finish is recorded as typed truncation and is not repaired by repeating the same call; empty content is likewise a typed invalid candidate and never reaches PlanIR.

2.2 Typed PlanIR and Live Verification

After decoding, a plan is a bounded directed acyclic graph with nodes from

$$\mathcal{N} = \{\text{OBSERVE}, \text{ASK}, \text{CONFIRM}, \text{ACT}, \text{RESPOND}\}.$$

Node IDs and evidence keys are unique; dependencies must exist and be acyclic; every node must reach exactly one terminal `RESPOND`. The default limits are 20 nodes and dependency depth 12. `OBSERVE` and `ACT` contain a live operation, constant JSON arguments, and a success key. `ASK` and `CONFIRM` wait for a correlated user event. `CONFIRM` also names the exact descendant action nodes it authorizes. `RESPOND` has one typed outcome—`completed`, `answered`, `refused`, `declined`, or `fail_safe`—and an explicit evidence set.

From T_t , PACT constructs a canonical, content-addressed capability snapshot. Each operation’s argument schema is

checked and executed with a Draft 2020–12 validator [JSON Schema, 2022]; nested objects, arrays, formats, numeric constraints, combinators, and additional-property rules therefore remain in the trusted boundary. Duplicate or malformed capability declarations fail closed. If trusted metadata labels an operation as `observe` or `act`, node kind must agree. PACT does not infer effects from operation names; absent metadata is represented as `unknown` and receives no kind-level guarantee.

2.3 Receding-Horizon Obligation Execution

Let $\sigma_t(n) \in \{\text{pending}, \text{waiting}, \text{satisfied}, \text{failed}\}$. The kernel selects the first topologically ready node

$$R_t = \{n \mid \sigma_t(n) = \text{pending}, \forall d \in \text{dep}(n), \sigma_t(d) = \text{satisfied}\}, \quad (3)$$

$$n_t = \min_{\prec} R_t. \quad (4)$$

It emits only n_t and then waits. Immediately before an external call, a fresh snapshot must have the same digest, contain the operation, preserve any known effect label, and accept a deep-copied argument object. Ask, confirmation, and observation evidence trigger a new short-horizon compile; after a successful action, a still-valid accepted graph may advance directly. Any successful action key is also placed in O_t , so a later horizon cannot silently forget a partially completed request.

Every ledger record stores source, status, event/call/node identifiers, operation, context/plan IDs, argument and schema digests, and optional authorizations. Values are isolated through a JSON round trip on both write and read, preventing nested caller mutation from rewriting history. Event IDs are fingerprinted: identical replay is idempotent, conflicting reuse and out-of-order input do not satisfy a node, and explicit external failure creates failure evidence and blocks that plan.

Confirmation is capability-conditional and scoped. For an operation marked confirmation-required, the verifier accepts either an ancestor `CONFIRM` authorizing that action node or a prior true confirmation record matching

$$(\text{schemaDigest}, \text{operation}, \text{SHA256}(\text{arguments})). \quad (5)$$

Boundary	Untrusted proposal	Local authority
Wire IR	JSON characters nodes, dependencies, outcome	strict unique-key decode frozen typed DAG + bounds
Capability	operation and arguments	live Draft 2020–12 schema
Transition	claimed next step	one ready node + matched event
Terminal	response text and class	evidence and obligation checks

Table 1: The model supplies semantic candidates; each executable property has a separate deterministic decision point.

The adapter consumes that scope after the matching action succeeds; it is not a general authorization for subsequent calls.

3 Assurance and Compute Bounds

We distinguish properties checked by code from assumptions about its inputs. Suppose the adapter is the only emission path, the live capability declaration is a valid schema, and hashing/JSON validation behave as specified.

Capability and schema confinement. If PACT emits external call $a_t = (o, x)$, then $o \in T_t$ and $x \models \text{schema}_{T_t}(o)$. The verifier establishes this for the accepted immutable plan; the emission guard repeats both membership and validation on a fresh digest-matched snapshot. Mutation or revocation between the two checks yields no call.

Grounded completion. For a typed `completed` response, PlanIR requires at least one action or carried evidence, the verifier requires every current action-success key and every key in O_t in the terminal evidence set, and carried keys must be successful external action records. At emission, every cited record must be external with status `success`. Hence no failed, user-authored, missing, or merely planned event can justify completion. Informational `answered` plans contain no action and cite their observation results; `refused` plans contain no external call; and `declined` requires a false confirmation record. Emergency `fail_safe` output is intentionally available even when a compiler or protocol failure prevents constructing failure evidence, but it never claims completion.

Transition confinement. Within the kernel, at most one node is waiting. Only a type-compatible event for that internal call can change it to satisfied. External events additionally match context, plan, node, operation, argument digest, schema digest, and a nonempty evaluator call ID. This establishes replay/idempotency properties *after* the A2A adapter has correlated the sole returned result; the protocol limitation below is material.

Why failures cannot escalate (proof sketch). Proceed by induction over processed events. Initially E_0 is empty and no node is satisfied. Registration of an unmatched event changes only its audit disposition. A matched input event can satisfy only the pending ASK/CONFIRM. A matched external event creates success evidence only when its explicit status is `success`; failure instead creates a distinct failure key and

marks the node failed. Duplicate and conflicting event IDs do not change the first stored record. Therefore none of these cases can manufacture a successful action record. The completion guard consults only such records, preserving the invariant.

Why prior actions cannot be forgotten (proof sketch). Within one goal, O_t grows after each satisfied ACT and is cleared only at a terminal or explicit safe termination. Every new compiler request receives O_t . The verifier rejects a carried key that is not a successful external action and rejects a completed terminal omitting any member of O_t . Thus, by induction over horizons, a reported completion accounts for all earlier effects. If a later obligation fails, the adapter instead emits partial-failure text.

Why confirmation is scope-limited (proof sketch). For an annotated capability, admission requires either an authorizing ancestor in the same immutable plan or a true ledger record whose schema, operation, and argument digests match. After a matching action succeeds, the adapter removes that individual authorization from the compiler-visible evidence projection. A record may intentionally hold multiple action scopes, but each exact scope is consumed independently; changing any tuple component makes it unusable.

Bounded inference and telemetry. For compiler request j , with repair indicator $r_j \in \{0, 1\}$,

$$N_j = 1 + r_j \leq 2. \quad (6)$$

Input length, conversation count, evidence count, candidate size, graph size, and graph depth have independent local ceilings. Provider rate-limit handling uses a finite retry limit $R_{\max}$ and cumulative wait limit $W_{\max}$, including proactive pacing and reactive header-aware backoff; exhaustion raises a typed error rather than waiting indefinitely. Across tool turns, provider usage is accumulated until the next text response and exported as `A2A_turn_metrics`. Competition-accounted usage is

$$B(q) = \sum_{j \in q} (p_j + c_j + h_j), \quad (7)$$

for reported prompt, completion, and thinking tokens. Call count, mean call time, model, cost, repair passes, and quota wait are reported alongside them; safe-error text includes the usage available from failed verification attempts.

3.1 Progress and Liveness Boundary

Safety does not imply that every request completes. For one accepted horizon that is not invalidated or replaced, suppose every emitted Ask/Confirm receives a compatible user event, every emitted external call receives one well-formed evaluator result, and the capability digest remains stable. Each such event changes exactly one waiting node to either satisfied or failed. Because the graph is finite and acyclic and no satisfied node is reissued, the horizon reaches its terminal response or a blocked state after at most $|\mathcal{N}|$ node emissions. Stable topological ordering makes the chosen trace deterministic when multiple independent nodes are ready.

These assumptions are intentionally stronger than the deployed environment. A user may never answer, a provider may exhaust its retry budget, an evaluator may return malformed

data, or a capability can be revoked. PACT preserves the outstanding obligation while waiting and otherwise chooses safe termination; it does not fabricate progress. Receding-horizon recompilation also prevents a global termination theorem: a sequence of new user events can keep changing the semantic goal. What is bounded is each compilation and each accepted-plan transition, not the number of possible interactions in an open conversation.

4 Engineering Evaluation and Limits

We evaluate the artifact’s enforceable mechanisms, not hidden outcomes. A clean local run on 17 July 2026 passed 330 tests plus three subtests in the repository; 168 tests directly target PlanIR, the verifier, compiler, ledger, runtime, A2A adapter, full JSON Schema guard, and bounded Cerebras retries. Representative adversarial cases include cycles and unreachable terminals; nested schema and contract-revocation failures; duplicate, stale, malformed, wrong-operation and wrong-digest results; deep mutation; confirmation replay; forgotten prior actions; exhausted repair; context cancellation; and metric preservation on safe errors. Generic non-automotive schemas exercise the same kernel. These checks are deterministic and reproducible with `uv run pytest -q`; they do not estimate CAR-bench reward or three-trial consistency.

Single-task public smoke. Using the submitted low reasoning setting, one run of public train task `base_0` earned reward 1.0. Its action, tool subset, tool execution, policy, and user-ending subrewards each reached 1.0. The trace read state, obtained shade confirmation and executed the adjustment, read weather, confirmed the rain-dependent operation, set the sunroof to 50%, and emitted evidence-grounded completion. No missing-tool, execution, or policy error occurred. Across three user turns it made seven compiler completion calls, including a local repair: 97,922 prompt, 3,972 completion, and 2,032 thinking tokens (103,926 by Equation 7), with 360,753.4 ms quota wait. Every observed provider response had `finish_reason=stop`. This validates one end-to-end evidence closure, not aggregate CAR-bench quality.

An earlier configuration canary explains the low default: at high reasoning, a 12,861-token prompt consumed the 4,096-token completion ceiling, of which 4,093 were thinking tokens, and returned `finish_reason=length` with no content. PACT now records that condition as truncation, preserves its usage, skips futile semantic repair, and fails safely.

The final runtime closure also passes Ruff and mypy over 16 production files; a local `linux/amd64` image starts and serves its A2A Agent Card. This establishes quality, packaging, and reachability—not digest-pinned public GHCR validation or live task performance.

Operational reproducibility. The release image contains only PACT dependencies and runs unprivileged on pinned bases. Environment variables configure inference and retry controls; the API key is supplied only at runtime. Audit records retain hashes and value-redacted issues. Context locks serialize turns, while cancellation deletes their state. Hashes are not signatures, and the ledger is not durable.

The assurance boundary has three important limitations. First, local verification cannot prove that the model correctly

interpreted natural-language policy, goal, or tool description. Typed citations and outcomes improve auditability but are not semantic proofs. Second, an operation whose trusted effect metadata is `unknown` can be structurally valid under either `OBSERVE` or `ACT`; confirmation guarantees apply only to explicitly annotated capabilities. Third, the current A2A tool-call payload does not let the agent attach its internal nonce to the evaluator’s outbound call ID. PACT therefore permits exactly one outstanding external obligation and checks returned operation plus a nonempty evaluator ID, but it cannot cryptographically distinguish an exact response from an evaluator replay of an identical call. This is a protocol trust assumption, not a proven property.

PACT’s central result is architectural: stochastic inference proposes meaning, while a small deterministic layer owns capability admission, evidence-bearing state changes, and terminal truth conditions. This separation supports compute-bounded execution without benchmark-specific workflows and makes both the achieved guarantees and the remaining semantic uncertainty explicit.

References

- [A2A Protocol Working Group, 2026] A2A Protocol Working Group. Agent2agent protocol specification, version 1.0. Linux Foundation, 2026. Accessed: 2026-07-17.
- [Cerebras Systems, 2026] Cerebras Systems. Structured outputs. Cerebras Inference Documentation, 2026. Accessed: 2026-07-17.
- [Geng *et al.*, 2025] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Generating structured outputs from language models: Benchmark and studies, 2025.
- [JSON Schema, 2022] JSON Schema. JSON Schema Core, draft 2020-12. Specification, 2022. Accessed: 2026-07-17.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 40599–40618, San Diego, California, 2026. Association for Computational Linguistics.
- [Leucker and Schallhart, 2009] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [Necula, 1997] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997.
- [Yao and others, 2023] Shunyu Yao *et al.* ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023.